

Modul Praktikum Flutter Fundamental

Pertemuan 3: Form Input dan State Management

Durasi	Level	Prasyarat
120–150 menit	Pemula	Menyelesaikan Pertemuan 1–2 (widget, layout, ListView, navigasi)

1. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Mengambil input pengguna dengan `TextField` dan `TextEditingController`.
2. Membuat form dengan validasi menggunakan `Form`, `TextFormField`, dropdown, dan checkbox.
3. Menjelaskan keterbatasan `setState` ketika data dipakai oleh banyak halaman.
4. Menerapkan state management dasar dengan `ChangeNotifier` dan paket **Provider**.
5. Membedakan penggunaan `context.watch` dan `context.read`.

2. Alat dan Bahan

- Flutter SDK, editor, dan emulator/perangkat dari pertemuan sebelumnya
- Proyek baru: `flutter create praktikum_3`
- Koneksi internet (untuk mengunduh paket *provider*)

3. Teori Singkat

Input dan form. `TextField` menerima teks; nilainya dibaca lewat `TextEditingController`. Untuk beberapa isian yang perlu divalidasi sekaligus, gunakan `Form` dengan `GlobalKey<FormState>` lalu panggil `validate()`. Setiap `TextFormField` memiliki properti `validator` yang mengembalikan pesan error, atau `null` bila valid.

State adalah data yang dapat berubah dan memengaruhi tampilan. Berdasarkan cakupannya:

Jenis state	Cakupan	Contoh alat
Ephemeral (lokal)	Satu widget	<code>setState</code> , checkbox tercentang
App state (global)	Banyak widget/halaman	<code>Provider</code> , <code>Riverpod</code> , <code>Bloc</code> ; daftar tugas, keranjang belanja

Provider menempatkan objek state (turunan `ChangeNotifier`) di atas pohon widget. Widget yang membutuhkannya cukup "berlangganan"; ketika model memanggil `notifyListeners()`, widget tersebut dibangun ulang otomatis.

Perintah	Fungsi	Dipakai di
<code>context.watch<T>()</code>	Membaca dan ikut membangun ulang saat data berubah	Dalam <code>build()</code>
<code>context.read<T>()</code>	Membaca sekali, tanpa membangun ulang	Dalam callback (<code>onPressed</code> , dll.)

4. Langkah Praktikum

Bagian A: Input Dasar dengan `TextField`

Ganti isi `lib/main.dart` dengan kerangka berikut, lalu tambahkan halaman `InputPage`.

```

import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 3',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const InputPage(),
    );
  }
}

class InputPage extends StatefulWidget {
  const InputPage({super.key});

  @override
  State<InputPage> createState() => _InputPageState();
}

class _InputPageState extends State<InputPage> {
  final _controller = TextEditingController();
  String _salam = '';

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Input Dasar')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: () {
                setState(() => _salam = 'Halo, ${_controller.text}!');
              },
              child: const Text('Sapa'),
            ),
            const SizedBox(height: 12),
            Text(_salam, style: const TextStyle(fontSize: 20)),
          ],
        ),
      ),
    );
  }
}

```

Checkpoint: mengetik nama lalu menekan tombol *Sapa* menampilkan sapaan di bawahnya. Perhatikan `dispose()`: controller harus dilepas agar tidak terjadi kebocoran memori.

Bagian B: Form dengan Validasi

Buat halaman baru `FormPage` (di file yang sama), lalu ubah `home`: menjadi `const FormPage()`.

```

class FormPage extends StatefulWidget {
  const FormPage({super.key});

  @override
  State<FormPage> createState() => _FormPageState();
}

class _FormPageState extends State<FormPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _email = TextEditingController();
  String? _jurusan;
  bool _setuju = false;

  @override
  void dispose() {
    _nama.dispose();
    _email.dispose();
    super.dispose();
  }

  void _ kirim() {
    if (_formKey.currentState!.validate()) {
      final jurusan = _jurusan ?? '-';
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Terdaftar: ${_nama.text} ($jurusan)'),),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Form Pendaftaran')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(
                labelText: 'Nama lengkap',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                (v == null || v.trim().isEmpty) ? 'Nama wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _email,
              keyboardType: TextInputType.emailAddress,
              decoration: const InputDecoration(
                labelText: 'Email',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                if (v == null || !v.contains('@')) return 'Email tidak valid';
                return null;
              },
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              decoration: const InputDecoration(
                labelText: 'Jurusan',
                border: OutlineInputBorder(),
              ),
              items: const [
                DropdownMenuItem(value: 'TI', child: Text('Teknik Informatika')),
                DropdownMenuItem(value: 'SI', child: Text('Sistem Informasi')),
                DropdownMenuItem(value: 'TE', child: Text('Teknik Elektro')),
              ],
              onChanged: (v) => setState(() => _jurusan = v),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

        validator: (v) => v == null ? 'Pilih jurusan' : null,
      ),
      CheckboxListTile(
        title: const Text('Saya menyetujui ketentuan'),
        value: _setuju,
        controlAffinity: ListTileControlAffinity.leading,
        onChanged: (v) => setState(() => _setuju = v ?? false),
      ),
      ElevatedButton(
        onPressed: _setuju ? _ kirim : null,
        child: const Text('Daftar'),
      ),
    ],
  ),
),
);
}
}

```

Checkpoint: (1) tombol *Daftar* nonaktif sampai checkbox dicentang; (2) mengosongkan nama atau mengisi email tanpa @ memunculkan pesan error merah; (3) bila semua valid, muncul *SnackBar*.

Bagian C: Mengapa Butuh State Management?

Pada Bagian A dan B, state hidup di dalam satu halaman (`setState`). Bayangkan aplikasi daftar tugas: halaman *daftar* menampilkan tugas, sedangkan halaman *tambah* membuat tugas baru. Kedua halaman perlu berbagi data yang sama. Mengoper data lewat constructor dan callback dari satu halaman ke halaman lain cepat menjadi rumit. Solusinya: pindahkan data ke satu objek bersama yang dapat diakses kedua halaman. Inilah peran **Provider**.

Bagian D: Aplikasi Daftar Tugas dengan Provider

Langkah 1. Pasang paket pada terminal di folder proyek:

```
flutter pub add provider
```

Langkah 2. Ganti seluruh isi `lib/main.dart` dengan bagian model dan entry point berikut:

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;
  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

  List<Tugas> get items => List.unmodifiable(_items);
  int get jumlahSelesai => _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}

void main() {

```

```

runApp(
  ChangeNotifierProvider(
    create: (_) => TugasModel(),
    child: const MyApp(),
  ),
);
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Tugas',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const TugasPage(),
    );
  }
}

```

Langkah 3. Tambahkan halaman daftar. Perhatikan pemakaian `context.watch` di dalam `build` dan `context.read` di dalam `callback`.

```

class TugasPage extends StatelessWidget {
  const TugasPage({super.key});

  @override
  Widget build(BuildContext context) {
    final model = context.watch<TugasModel>();

    return Scaffold(
      appBar: AppBar(
        title: Text('Tugas (${model.jumlahSelesai}/${model.items.length})'),
      ),
      body: ListView.builder(
        itemCount: model.items.length,
        itemBuilder: (context, i) {
          final t = model.items[i];
          return ListTile(
            leading: Checkbox(
              value: t.selesai,
              onChanged: (_) => context.read<TugasModel>().toggle(i),
            ),
            title: Text(
              t.judul,
              style: TextStyle(
                decoration: t.selesai ? TextDecoration.lineThrough : null,
              ),
            ),
            trailing: IconButton(
              icon: const Icon(Icons.delete),
              onPressed: () => context.read<TugasModel>().hapus(i),
            ),
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const TambahPage()),
          );
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}

```

Langkah 4. Tambahkan halaman untuk menambah tugas (halaman terpisah, tetapi memakai model yang sama):

```

class TambahPage extends StatefulWidget {
  const TambahPage({super.key});

  @override
  State<TambahPage> createState() => _TambahPageState();
}

class _TambahPageState extends State<TambahPage> {
  final _controller = TextEditingController();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _simpan() {
    final judul = _controller.text.trim();
    if (judul.isEmpty) return;
    context.read<TugasModel>().tambah(judul);
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tambah Tugas')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              autofocus: true,
              decoration: const InputDecoration(
                labelText: 'Judul tugas',
                border: OutlineInputBorder(),
              ),
              onSubmitted: (_) => _simpan(),
            ),
            const SizedBox(height: 12),
            ElevatedButton(onPressed: _simpan, child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}

```

Checkpoint: (1) menambah tugas dari halaman kedua langsung muncul di halaman daftar; (2) mencentang tugas mengubah judul AppBar (misalnya 1/3) dan mencoret teks; (3) ikon sampah menghapus tugas.

5. Latihan Mandiri

1. Tambahkan validasi pada TambahPage: judul minimal 3 karakter dan tampilkan pesan error (ubah ke TextFormField + Form).
2. Tambahkan method `hapusSelesai()` pada `TugasModel` dan tombol di AppBar untuk menghapus semua tugas yang sudah selesai.
3. Tampilkan `SnackBar` "Tugas ditambahkan" setelah tugas disimpan.
4. Bila daftar kosong, tampilkan teks "Belum ada tugas" di tengah layar.

6. Tugas

Buat aplikasi **Daftar Belanja** dengan ketentuan:

- Halaman *Form Tambah*: nama barang (wajib), jumlah (wajib, angka lebih dari 0), dan kategori (dropdown), dengan validasi pada setiap isian.

- Halaman *Daftar*: menampilkan semua barang, dapat dicentang sebagai "sudah dibeli" dan dihapus.
- State disimpan pada satu `ChangeNotifier` dan dibagikan dengan `Provider`; `AppBar` menampilkan jumlah barang yang belum dibeli.
- Kumpulkan: tangkapan layar kedua halaman (termasuk tampilan pesan error validasi) dan berkas `main.dart` (atau tautan repositori).

7. Rubrik Penilaian

Komponen	Bobot
Bagian A–D berjalan (checkpoint)	35%
Latihan mandiri	20%
Tugas Daftar Belanja (validasi, state management, kelengkapan)	35%
Kerapian kode dan penamaan	10%

8. Pertanyaan Refleksi

1. Mengapa `TextEditingController` harus di-*dispose*?
2. Kapan cukup memakai `setState`, dan kapan sebaiknya beralih ke `Provider`?
3. Apa yang terjadi bila `notifyListeners()` lupa dipanggil? Mengapa?
4. Mengapa di dalam `onPressed` kita memakai `context.read`, bukan `context.watch`?

9. Troubleshooting Umum

Masalah	Solusi
Error: Could not find the correct Provider	Pastikan <code>ChangeNotifierProvider</code> berada di atas <code>MaterialApp</code> (di dalam <code>runApp</code>)
Tampilan tidak berubah setelah data berubah	Periksa apakah <code>notifyListeners()</code> dipanggil dan widget memakai <code>context.watch / Consumer</code>
Package provider tidak ditemukan	Jalankan <code>flutter pub get</code> dan restart aplikasi (bukan hot reload)
<code>validate()</code> tidak bereaksi	Pastikan key: <code>_formKey</code> terpasang pada <code>Form</code> dan isian memakai <code>TextFormField</code>
Layar tertutup keyboard/overflow	Gunakan <code>ListView</code> atau <code>SingleChildScrollView</code> sebagai induk form

10. Referensi

- Membuat form dan validasi: docs.flutter.dev/cookbook/forms
- Pengantar state management: docs.flutter.dev/data-and-backend/state-mgmt
- Paket provider: pub.dev/packages/provider